I. Text Summarization using Bi-LSTM with attention & pre-trained model Bart

Aarchi Gandhi, A00025390

Deep Learning Applications

University of Roehampton, London UK

***Abstract*—** The research in the field of Text Summarization (TS) has been through many different transformations in recent years with advances in deep learning neural networks and pre-trained language models (PLMs). The report includes understanding of 4 research papers from the last 5 years in the field of Text Summarization. This report focuses on producing an automated text summarization system especially for news articles. Two different models have been implemented for the task on **CNN/DailyMail Dataset of version 3.0.0** from Hugging Face Library: A **Bi-LSTM with Attention** architecture and **BART (Bidirectional and Auto-Regressive Transformers)** model. The model has been deployed using **Gradio** for its easy interaction and testing. All the steps of accessing the app are mentioned in the report. This report depicts the use of advanced NLP techniques of summarization tasks, leading towards potential future research for scientists and an easy, simpler application for users to summarize their documents.

Keyword: Summarization, NLP, Encoder-Decoder Architecture, BART, Gradio, Hugging Face

II. Introduction

The process of Text Summarization is used to convert the long articles and documents into short and concise summaries that include all the important information and is crucial in a world where being time efficient is the topmost skill to be productive. There are two main methods of generating summaries. Extractive summarization and Abstractive Summarization. Extractive models generate summaries from the same sentences as in the input articles or original text directly. Abstractive summarizing models tries to understand the context of the input sequence and then generates a more concise summary by paraphrasing the original text [1].

Initially, my project started with developing a Bi-LSTM with Attention model. After training, to enhance the performance of the model and summarizer, I used Transformer based architecture which is considered the most efficient for tasks like summarization. Therefore, I transitioned to using BART (Bidirectional and Auto-Regressive Transformer) [2]. The goal of this project is:

- Fine-tune a pre-trained BART model on the CNN/Daily Mail dataset to perform summarization.
- Compare the performance of BART and Bi-LSTM with Attention.
- Deploy the model using Gradio for user testing.

This task is significant in various real-world applications including analyzing and summarizing financial reports, legal documents, and most importantly, scientists and students can efficiently find necessary details from long documents and research papers [3].

III. Background and problem definition

With the increasing number of textual resources on the internet, extracting and utilizing the information has become a very complex and time-consuming task which may lead to big problems where time constraint and accurate content is a necessity such as in judicial systems while analyzing legal documents. The automatic text summarization task will help find the most crucial information in the long documents and help save a lot of time.

Therefore, the major part and motivation for further researching the topics are its applications in the real-world. Some of them include summarizing books and novels, media companies use summarization methods to analyze the messages and posts on their platforms like Instagram, Twitter; one can summarize others feeling or views by doing sentiment analysis like amazon reviews, surveys, etc., it can help in generating a headline for the newspapers, summarizing legal or financial documents and also research papers for easy understanding of a subject [4].

Moreover, TS is an old task and has attracted a lot of researchers in recent years. Below are 4 research papers from the year 2021-25 that inspired me to delve into the topic of text summarization.

- A team of researchers, majorly from Dhaka, worked together to do a comprehensive survey on the progress, process and challenges in Automatic Text Summarization. The paper reviewed extractive and abstractive approaches with traditional methods like TF-TDF, graph-based, seq2seq, attention mechanism, transformers like GPT and various techniques for preprocessing and feature extraction. It also offers a deep understanding of the evaluation metrics such as ROUGE and BLEU and challenges like interpretability and quality of the summarized text. [4]

- In 2022, Mengli Zhang and the team moved their focus from extractive summarization to abstractive TS using Deep Learning architectures. They reviewed several datasets used for training and testing. They applied summarization systems of various architectures such as Transformers, RNN with reinforcement learning, BERT, PEGASUS and BART on CNN/Daily Mail dataset, evaluated using ROUGE and the scores when trained on BART were 44.16 for rouge-1 and 41.11 for rouge-L. their report highlighted some future inspiration to generate personalized summary and cross-language summarization [1].

- The task is not limited to documents and research papers. A team of researchers from India implemented this task for *biomedical transcripts* using a deep dense *LSTM-CNN* architecture. After training the MT Samples data which consists of 40 different medical fields such as allergies, diet, nutrition, etc., their approach resulted in an average ROUGE score of 93.5% for single document summary [5].

- Another research was done on using BART for summarizing text by a team of students from India on CNN/Daily Mail Dataset. The core design of the model consists of 12 encoder layers and 12 decoder layers with multi-head self-attention and feed-forward network in each layer. After tokenization, it had 1024 and 128 tokens for maximum input and summary length. They evaluated using ROUGE scores which measure the overlap of unigram (rouge-1), bigram (rouge-2) and longest common subsequences (rouge-L) in the summary, calculated as 0.36, 0.23 and 0.26 respectively [6].

IV. PROPOSED MODEL DETAILS

A. *Bi-LSTM with Attention Mechanism (First Approach)*

Overview of Bi-LSTM: Bi-directional Long Stort Term Memory is a type of Recurrent Neural Network (RNNs) which is used to process long sequences in both **forward** and **backward** directions. This allows the model to capture the meaning from beginning and end of a sentence. It is efficient because LSTM are designed to mitigate the problem of *vanishing gradient* which in turn helps in capturing long-range dependencies in text [7].

Attention Mechanism: The attention mechanism is used in models to '**highlight'** on the important part of the input sequence by computing a weight for all the hidden state for every word.

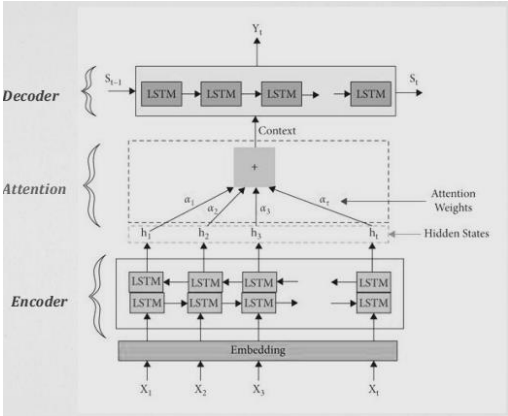

Fig 1. Bi-LSTM model architecture (image source)

From the fig.1, it is observable that after the input will preprocess into tokens like x1, x2, ..., xi, the encoder will read the tokens in both directions and create a hidden state (h1, h2...) for each word. Attention weights will provide weight for each hidden state which will show what part is more important and generate a context vector to pass to the decoder and the decoder will again put the sequence together and generate summary based on the weighted states from the encoder.

However, Bi-LSTMs can take a long time to train due to their sequential nature and capture long dependencies if the text is too long [3].

B. *BART Model (Final Approach)*

BART is a sequence-to-sequence model whose architecture follows encoder-decoder framework.

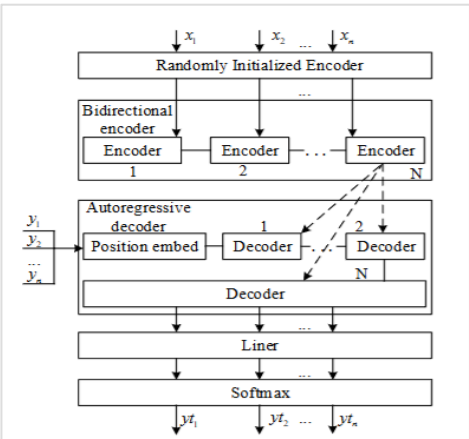

Fig.2 BART architecture (image source)

As per the fig.2, the input tokens x1, x2, …, xn will first pass through randomly initialized encoder which can also be referred to as Embedding layer to convert tokens into dense vectors. It then moves toward a bi-directional encoder where it learns the tokens context. The autoregressive decoder will generate summary by each token in sequential format with positional embedding which will help in maintaining word order and linear and SoftMax will convert the output into predictions.

The advantages of using BART over Bi-LSTM are that it is built using transformers, meaning that the model processes multiple inputs at the same time which makes it faster. It is better at understanding the meaning of words in the sequence which are not next to each other. According to the researchers, BART has higher performance than other models in tasks like text summarization.

V. DATA SELECTION

In this project, I have used the CNN/Daily Mail Dataset of version 3.0.0 from the Hugging Face (HF) Datasets library. The 3.0.0 version consists of a fully anonymized version of the dataset which means that it does not have an id column and includes two columns Article and Highlights. It is considered as one of the best and most suitable datasets for natural language processing, especially for summarization tasks because of its **volume** and the **structure** of the data having article-summary pairs. The dataset also has its standardized training, validation and testing split.

Rationale behind using HF library is that it makes accessing and reproducing the dataset for future use easier. One of the major concerns while manually loading dataset was data loss. Out of 300,000 only 4775 samples were loading due to parsing error. Therefore, to make sure that the model remains effective and robust, I believed using HF library to be a good choice for the project.

The structure of the dataset is as follows: two columns, **Article** and **Highlights**. It has around 300,000 unique news articles and summary pairs with average article token count of **781 words** and highlights of **56 words**. The train, validation and test split are 287,113 training samples, 13,368 validation samples and 11,490 test samples of the dataset.

VI. CODE DOCUMENTATION (PREPARING, MODEL DEFINITION, TRAINING & FINE-TUNING)

Here, I have recorded all the details I have used during the task in both the proposed methods.

A. *Bi-LSTM with Attention*

- Imported necessary libraries and then loaded the cnn_dailymail dataset of version 3.0.0 from the Hugging Face datasets library. After converting to dataframe, I moved forward to clean and preprocess the data.
- For cleaning the text, I created a function called 'text_cleaned', which removed any HTML tags (<b, i>), special characters, converted to lower case and applied lemmatization which means converting any word to its root word so that the model can learn efficiently. For instance, the word 'running' would be converted to 'run'.
- Due to the very large size of the dataset and my computational constraints, I have used a sample subset of the dataset with 1000 samples for training, 500 for validation and 200 other samples for testing.
- Before moving forward to pre-processing, I have assigned the parameters for training the model as follows: maximum article words to 200, summary words to 40, vocabulary size to 5000, embedding dimension of 32 for

vector representation, 32 LSTM units in the layers, a batch size of 16 and epoch or number of iterations to 10.

- For pre-processing and preparing the data for the model, the cleaned data was then tokenized where two tokenizers were created for two columns and were set to the limit of vocabulary size, which was set to 5000 words. Any word which is outside this size will be considered as Out-of-vocabulary (OOV) and are replaced with their tokens. The tokens then convert the text into numerical sequence as padding can only be applied to numerical data to ensure that the text is set to fixed lengths. The longer sentences will be truncated or cut whereas the short sentences would be padded with 0 in the end.
- The next step consisted of **defining the model architecture**, the very first task was to assign the proper encoder and decoder inputs. I have applied a shifting operation here which creates a **teacher-forcing** approach where the first input becomes a start token, making sure that each predicted token becomes input for the next word. Then I created a function named '**seq2seq_model'**, which included an *encoder* consisting of input layer, embedding layer, Bi-lstm layer, attention layer (weighted sum) and *decoder* consisting of decoder inputs, embedding layer and LSTM layer.
- The dense output layer used **SoftMax** activation function and **dropout** to prevent overfitting.
- The last task here is compiling the model with appropriate optimizer, loss function and metrics.
- Before training, I fine-tuned the model by defining callbacks including early stopping and saving the model via model checkpoint. I further implemented rouge score and tried using different optimizers to see how the model performs with different optimizers such as Adam, sgd and adagrad. I have created an experiment function which computes rouge scores for all three training with three different optimizers.

B. BART Model

The process is divided into two sections. Firstly, I have trained my model and saved it in drive and then in the other section, I loaded my saved model from the drive to use for testing.

- The first step is to install the necessary libraries like transformers, datasets, gradio, and so on.
- I have imported libraries namely, BartTokenizer which will tokenize the text and feed into the model, BartForConditionalGeneration which is a BART model class for summarization tasks, TrainingArguments and Trainer for fine-tuning and training the model and DataCollatorForSeq2Seq for preparing the correct batch sizes for faster training and less memory usage.
- Loading the CNN/Daily mail dataset that contains articles and their summaries.
- The next step is importing the BART model (Bart-large-CNN), loading the tokenizer using Bart Tokenizer and loading the pre-trained model.
- After this, I created a function that will prepare all the pairs for training by converting **articles** to tokens and limit them to **512**, highlights to tokens limiting to **128** tokens and added a **summary** token called 'labels.
- Applied the pre-processing to the entire dataset and extracted a smaller subset of the datasets like **1000 training** examples and **200 validation** examples for faster training.
- Then, I set all the training parameters including learning rate, batch size, data collator and after training, saved the fine-tuned model for future use.
- The second section of the code includes loading the fine-tuned model from the drive. For proper accessibility, I created an if statement specifying to load the model and if it does not exist, it would still work and use the original pre-trained 'facebook/bart-large-cnn' model.
- Then, I created a summarize_article function, which had all the details like input length, input, maximum and minimum length, early stopping, number of beams to 4, meaning, the model will look for 4 most likely next words in the text and length penalty to 2.0 which would tell the model to generate a concise summary and improve its quality.

VII. MODEL TESTING

As mentioned previously, I have implemented three experiments and computed their rouge scores. First, let us understand what ROUGE means.
It stands for Recall-Oriented Understudy for Gisting Evaluation and is used to measure the overlap or n-grams between the original and predicted summary. It has three main measures, namely, rouge-1, rouge-2 and rouge-L.
Rouge-1 measures the overlap of unigrams which compares every single word in original and predicted summary.
Rouge-2 measures the overlap of bigrams, meaning, compared two consecutive words in a sequence.
Rouge-L finds the longest common sequence between the original summary (highlights) and the predicted summary.

The rouge scores I achieved for all the experiments are as follows:

Experiment Name	ROUGE-1	ROUGE-2	ROUGE-L
Adam Optimizer	0.405875	0.265256	0.405875
SGD Optimizer	0.399375	0.259103	0.399375
Adagrad Optimizer	0.405375	0.265000	0.405375

Fig.3 rouge scores table

From the above table, it is visible that the best optimizer for these experiments with 10 epochs is Adam optimizers with the scores of 40.58, 26.52 and 40.58 and the training accuracy I achieved was 34.45%. By increasing the number of samples and epochs, the model would perform quite well on a Bi-LSTM with Attention Mechanism.

For BART approach, I only trained the model for 1 epoch and got a similar training accuracy. Therefore, using transformer-based models is more efficient.

VIII. MODEL DEPLOYMENT

For accessing and testing the model, I deployed the model using **Gradio** which is an **open-source python library** used for creating simple interactive web interfaces.

Deployment guide:

Firstly, I imported libraries including transformers, datasets and gradio and created a function called **summarize_article** and created an interface object by passing all the parameters of the model into the interface and then launched it by calling the object. When you run the interface, colab will generate a public URL to access the web application and generate summaries.

However, the gradio link only works when the code file is running, therefore for easily using the summarizer, I deployed my model on Hugging Face Spaces. It is a platform which provides a space to publish or deploy our models for the public to use without running the code again and again.
To create that,

- I created my account on Hugging Face and then a Space on Hugging face and filled all the details like space name (text_summarizer), SDK (Gradio) and visibility (public).
- Then, I uploaded my files: app.py and requirements.txt.
- The space automatically generates a link in a few minutes, and people can use the link to get a summary without running the code all over again.
- One important thing to remember is that, if the space is inactive for a few days, the space will go into sleep mode. All one must do to get it running is click on the 'restart space' button on the screen. The space will start running again for summarization.

The steps to get a summary:

- Click on the link below to go to another tab which will take you to the application main page.
- If the app is not running, restart the space.
- When the app starts running, you will see two text boxes. One is for entering input and another on the right-hand side is for the output or summary.
- Enter your input article and then click on '*Submit'*. The output will be obtained on the textbox with Summary written on it.
- If you want to generate another summary, click on '*clear'* and enter your next article.

The link to my deployed web app is:
https://huggingface.co/spaces/AarchiGandhi/Text_Summarizer

IX. Results and analysis

This section discusses the results obtained from the two summarization methods used: Bi-LSTM with Attention mechanism and BART. I have analyzed the performance of each model and discussed their applications in the real-world.

The first model was trained for 10 epochs using a subset of CNN/Daily Mail dataset due to computational constraints. The Adam optimizer surpassed SGD and Adagrad in terms of accuracy and rouge scores. It showed the rouge-L score of 40.58 which indicates some kind of similarity between the original and predicted summary.
My analysis of my approach was that the model has its own limitations such as low accuracy and rouge scores which were also the results of limited training samples and number of times the model was trained. Thus, taking into account the number of constraints, the Adam optimizer emerged as a suitable choice for this model which provided reasonable performance for summarizing a text document.

After fine-tuning the BART model, I evaluated it on a few samples of news articles. It provided concise and relevant summaries, and its tone was very much like the human-written summaries.
Although, there were a few challenges that I faced like the size of the model. Bart is a large model and so I applied gradient accumulation to take larger batch sizes by reducing memory overflow.
One such example is as follows:
Original Summary input: The enormous quantity of text makes it challenging for users to obtain the key information and knowledge. Automatic text summarization can alleviate this problem by providing reliable summaries for massive text documents. During the last decade, significant achievements have been made in text summarization. We conduct this survey to explore what research community is focused on, the application scenarios of summarization, the state-of-the-art techniques and methods, and to analyze the challenges and future direction. We summarize that incorporating with natural language processing, previous text summarization research applied knowledge-based methods, graph-based methods, statistical learning methods, and deep learning methods.

Predicted summary: The enormous quantity of text makes it challenging for users to obtain the key information and knowledge. Automatic text summarization can alleviate this problem by providing reliable summaries for massive text documents. We conduct a survey to explore what research community is focused on and the application scenarios of summarization.

X. Conclusion

This project demonstrates the use of RNNs and transformer-based models like BART for the task of text summarization. The model was fine-tuned on CNN/Daily Mail Dataset and deployed by Gradio for simple yet effective testing. As compared to Bi-LSTM with Attention, BART gives better results and performance for summarization. This tool will definitely save users a lot of time and effort in reading and understanding all the long documents.

There are more potential future improvements that can be made, some of which include further exploring better and advanced architectures for better accuracy, implementing enhanced features that will allow users to adjust their input length. As of now, the model's ability is only to summarize news articles, which can be further enhanced to summarize all kinds of articles and also improving the model to produce more abstract summaries rather than extractive.

Moreover, the fine-tuned BART model deployed on Hugging Face Spaces gives a user-friendly interface for summarizing news articles.

XI. References

[1] G. Z. W. Y. N. H. W. L. Mengli Zhang, "A Comprehensive Survey of Abstractive Text Summarization Based on Deep Learning," *Computational Intelligence and Neuroscience,* vol. 2022, no. 1, 2022.

[2] Facebook, *huggingface.co/facebook/bart-large-cnn.*

[3] H. &. Y. P. &. J. Z. Zhang, "A Systematic Survey of Text Summarization: From Statistical Methods to Large Language Models," arXiv, 2024.

[4] A. A. L. K. N. S. C. D. M. H. M. M. K. M. F. Mridha, "A Survey of Automatic Text Summarization: Progress, Process and Challenges," *IEEE Access,* vol. 9, 2021.

[5] P. P. S. B. M. &. S. K. Bedi, " Extractive text summarization for biomedical transcripts using deepdense LSTM-CNN framework," *Expert Systems,* vol. 41, no. 7, 2024.

[6] V. D. R. J. D. A. S. B. Pragya Tewari, "Transformer-Based Abstractive Text Summarization: A Comparative Study Using BART," in *International Conference on Innovative Computing and Communication,* 2025.

[7] P. V. S. M. O. S. N. S. T. Watanangura, "A Comparative Survey of Text Summarization Techniques," *SN Computer Science,* vol. 5, no. 1, 2023.